

Technical Note — Memory & State Management for a Personal Assistant

Zynxis Agentic AI Internship — Week 3 Author: Faozan Mujtaba

1. Problem

A single LLM call has no memory of anything outside its context window. A useful personal assistant needs to (a) hold a coherent conversation turn-to-turn, and (b) recall facts stated far earlier — potentially in a previous session — without re-sending the entire conversation history on every request (which would be slow, expensive, and eventually exceed the context window).

This calls for two distinct memory tiers with different retention and retrieval strategies.

2. Architecture

```
graph TD
    UserQuery[User query] --> LTMRecall[LongTermMemory.recall(query) -- ChromaDB similarity search --> top-k past facts]
    LTMRecall --> STMAsMessages[ShortTermMemory.as_messages() -- last N turns, verbatim]
    STMAsMessages --> LLM[LLM]
    LLM --> Answer[answer]
    LLM --> STMAdd[ShortTermMemory.add(...) (buffer updated; oldest turn evicted past capacity)]
    STMAdd --> LTMRemember[LongTermMemory.remember(...) (new fact embedded and stored permanently)]
    LTMRemember --> LTMRecall
```

[system prompt] + [recalled memories] + [short-term buffer] + [new query] --> LLM --> answer

Short-term memory (`memory.ShortTermMemory``) is a plain Python list capped at the last N turns (N=3 in the demo, i.e. 6 messages). It is fast, exact, and free — no embedding or database round-trip — which is the right trade-off for "what did we just say two messages ago."

Long-term memory (`memory.LongTermMemory``) wraps a ChromaDB `PersistentClient``. Every user statement is embedded (via ChromaDB's default sentence-embedding function) and stored with a timestamp. On each new turn, the query is embedded and the top-k most similar stored memories are pulled back and injected into the prompt as a "Relevant memories" system message. Because this is similarity-based rather than a fixed window, a fact from turn 1 can still be recalled at turn 50 as long as the current query is semantically related to it.

3. Why ChromaDB (over Pinecone)

Pinecone requires a hosted account, an API key, and network calls for every read/write. ChromaDB runs embedded, in-process, persisting to a local directory (`./chroma_store/`) with zero external dependencies or cost. Given Week 1/2 already worked around regional unavailability of some hosted free tiers (Gemini, OpenAI), a fully local vector store removes an entire class of availability risk while satisfying the same "vector DB long-term memory" requirement. The trade-off is that ChromaDB's local store doesn't horizontally scale across machines the way a hosted index does — irrelevant at this project's scale (a single user's conversation history).

4. Design decisions & trade-offs

- **Storing every user message, not just "important" ones.** A production system might have the LLM decide what's worth remembering (e.g. a `remember_fact`` tool call). We store everything here for simplicity and guaranteed recall correctness; the trade-off is accumulated conversational noise over a long-running deployment, which a relevance filter would fix.
- **Fixed-size short-term window, no summarization.** Evicted turns are dropped rather than summarized before leaving the buffer. Cheaper (no extra LLM call per eviction) at the cost of losing exact phrasing — only the standalone fact re-enters context via long-term recall, not the surrounding nuance.
- **Recall injected as a labeled system message, not merged into chat history.** This lets the model correctly attribute where a fact came from and avoids fabricating a false conversational history.

5. Demo result

`assistant.py``'s demo runs 6 turns with a short-term window of 3. The user's favorite programming language, stated in turn 1, is asked about again in turn 6 — well after the short-term buffer has evicted it. Long-term recall correctly retrieves the original

statement via similarity search, and the model answers correctly, citing the earlier introduction. This confirms the two-tier design does what it's meant to: near-term coherence from the buffer, durable recall from the vector store.

6. Possible extensions

Tag memories by category (preference/fact/task) for targeted recall; add expiry so stale facts don't outlive their relevance; summarize evicted turns instead of dropping them if conversational nuance needs to survive long-term.